

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

Task 1: Project Phase-I Summary

This project looks at credit card fraud detection, framed as a binary classification problem: given a set of transaction-level features, predict whether an individual transaction is fraudulent or legitimate. The target variable is Class, where 1 is fraud and 0 is a legitimate transaction.

The dataset used is the Credit Card Fraud Detection dataset, sourced from Kaggle and originally by the Machine Learning Group at Universite Libre de Bruxelles (ULB), covering transactions made by European cardholders over two days in September 2013. The dataset contains 284,807 records across 31 columns: 30 features (Time, V1-V28, and Amount) plus the target Class. V1-V28 are anonymised, PCA-transformed components with no disclosed real-world meaning, while Time and Amount are the only two features in their original form.

Phase 1 saw many key data-quality characteristics. The dataset contained 1,081 duplicate rows, which were removed to prevent data leakage between training and test sets. Amount was severely right-skewed (skewness of 16.979), which was addressed through a log transformation, reducing skewness to 0.161. No missing values, invalid values, or inconsistent entries were found. The most significant characteristic of the dataset is the severe class imbalance. Only 492 of the original 284,807 transactions (0.17%) are fraudulent, a proportion that stayed unchanged after cleaning (473 of 283,726, 0.167%).

This problem is suited to an LCS-based solution for many reasons. LCS systems evolve interpretable, human-readable rule sets rather than opaque decision boundaries, which is needed in a fraud-detection context where decisions may need to be explained to stakeholders such as banks or regulators. In addition, Phase 1's correlation analysis showed that only a few features (V17, V14, V12, V10, V11, V4) had linear signals for predicting fraud, whilst many features contribute little to nothing. That is a pattern LCS's rule discovery mechanism is suited to exploit, since it learns to condition rules on relevant feature ranges rather than needing all 30 features to contribute. The dataset's structure (fully numeric, tabular, binary-labelled) is also compatible with standard LCS implementations such as eLCS.

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

Task 2: Original LCS System on raw Dataset

Yaacoub:

Task 2: Original LCS System on Raw Dataset (Results)

An original, unmodified eLCS system (scikit-eLCS implementation) was run on the raw dataset prior to any cleaning or transformation. Due to the computational cost of LCS rule evolution on the full 284,807-row dataset, a stratified subsample of 15,000 rows was used, preserving the original ~0.17% fraud rate (21 fraud cases in the 12,000-row training set; 5 in the 3,000-row test set). This subsampling, along with separating features and the target label into NumPy arrays, was the only processing applied. No cleaning, scaling, or transformation was done, in line with the requirement to establish a baseline on raw data.

The model was trained using eLCS's default parameters (learning_iterations = 10,000, N = 1,000, p_spec = 0.5, nu = 5, chi = 0.8, mu = 0.04, theta_GA = 25) with no modification. Training took 39.41 seconds.

Baseline results:

Metric	Value
Accuracy	0.9983
Balanced Accuracy	0.5000
Precision	0.0000
Recall	0.0000
F1-score	0.0000

Although accuracy is very high, it's misleading given the class imbalance. A classifier that predicts legitimate for every transaction would have similar accuracy. The balanced accuracy of 0.5, alongside zero precision, recall, and F1-score, shows the original LCS system failed to identify any fraudulent transactions in the test set, consistent with the small number of fraud examples in the raw, unprocessed training data (21 cases). This suggests the system didn't encounter enough fraud examples to evolve rules capable of distinguishing fraud from legitimate transactions. This result supports the preprocessing and improvement work done in tasks 3 and 4.

Even though accuracy seems high (99.83%), this is very misleading given the class imbalance. The balanced accuracy of 0.5 and zero precision, recall, and F1-score confirm the original LCS system failed to detect any fraud in the raw, unprocessed data, motivating the preprocessing and improvements in Task 3 and 4.

Matthew:

Task 2 Results: Original LCS System on Raw Dataset

A baseline eLCS system, provided via scikit-eLCS, was deployed against the unadulterated dataset without prior cleaning or transformation. Due to the high computational costs of evolving LCS rules on the entire 284,807 row dataset, a 40,000 row subset was sampled (using the exact per-class counts in order to maintain the original fraud rate of 0.1727%), resulting in 55 cases of fraud among 32,000 training rows and 14 cases among 8,000 test rows. Other than this sampling, the only other processing was splitting features and the target label into numpy arrays. There was no cleaning, scaling, or transformation as a baseline must be established from raw data.

The model was trained using the default parameters for eLCS - learning_iterations = 10,000; N = 1,000; p_spec = 0.5; nu = 5; chi = 0.8; mu = 0.04; theta_GA = 25 - without any modifications. The total time for training was 36.33 seconds.

Baseline results:

Metric	Value
Accuracy	0.9982
Balanced Accuracy	0.5000
Precision	0.0000
Recall	0.0000
F1-score	0.0000

This is not, however, as impressive as it looks. Due to the imbalanced class nature of the problem, a simple classifier which guessed every transaction was legitimate would attain an almost identical score. A balanced accuracy of 0.5 along with precision, recall and F1-score of 0 is evidence that the original LCS in this configuration failed to predict any of the fraud cases in the test set. This is unsurprising given that the amount of fraud examples in the training data not prepared by undersampling consists of only 55. There is arguably simply not enough of these for the system to learn enough rules to discriminate them from the legitimate cases.

An original, unmodified of eLCS, was applied to the raw (uncleaned and untransformed) dataset. To sub-sample instead of relying on train_test_split stratify parameter, it is an explicitly computed class specific sample sizes using the raw fraud rate of 0.1727%. Each class was sampled independently, then merged and shuffled, thus directly controlling the fraud rate of the sample rather than doing it indirectly to preserve class distribution.

The baseline used managed to get no detections. The Balanced Accuracy is at 0.5000, while precision, recall and F1-score are 0.0000. While the Accuracy score appears high at 99.82%, it is misleading as a degenerate classifier simply outputting "legitimate" for every single input would have achieved near identical accuracy despite detecting 0 frauds.

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

Task 3: Data Preprocessing and Feature Engineering

Yaacoub: Missing values, invalid values, inconsistent categories, and data types were addressed in Phase 1, where none were found. The 1,081 duplicate rows identified in Phase 1 are removed here before splitting, to prevent identical transactions appearing in both training and test sets. The dataset is split into training and test sets before any scaling or resampling is applied, ensuring no information from the test set influences preprocessing decisions. Outliers in Amount, investigated using the IQR method in Phase 1, was retained rather than removed, as they represented genuine high-value transactions rather than invalid data, and fraud was only marginally overrepresented among them.

Task 3: Data Preprocessing and Feature Engineering (Results)

This preprocessing is expected to improve LCS performance in many ways. eLCS's rule-discovery process depends on seeing enough examples of the minority class to evolve rules that cover it. Task 2's baseline showed the model failed to detect any fraud when trained on the raw, imbalanced data. SMOTE addresses this by providing a much larger and balanced set of fraud examples to learn from. Scaling Time and Amount to match the standardised range of the PCA components should also help these features contribute more evenly to rule matching, rather than being overshadowed by features on different numeric scales. Finally, the reduced feature set is expected to shrink the LCS search space, potentially allowing rules to specialise more effectively around the small number of features known to carry the strongest fraud signal.

Matthew:

In phase 1, data quality was evaluated; it yielded 0 missing values, invalid values, inconsistent categories and data types. 1,081 duplicate row was then removed before the split to avoid the same transactions during both train and test sets. Also in phase 1, the Outliers were examined with Z-scores ($|z| > 3$) in Amount which revealed 4,063 transaction (1.43%). Only 11 of these 4,063 were frauds. This proportion is in line with the fraud rate of the dataset, implying that an exceptional transaction amount is not a good signal for a fraud. Thus, it was decided that outliers would not be removed because excluding them risks removing valid frauds.

Task 2's exact-per-class-count subsampling method was reused, keeping the original fraud rate and reducing to 40,000 rows. The split for training/testing was done right after sub-sampling and prior to any fitted preprocessing step to make sure test information would never contaminate the training phase.

Amount was strongly right-skewed; Yeo-Johnson power transformation (Yeo & Johnson, 2000) was used as it allows transactions of amount 0, a problem for the normal log transform. Additionally, a Robust Scaler was used to scale Time as its median/IQR-based scaling with the decision to keep rather than removing the outliers. Both transforms were fitted only on the training set, and the subsequent test sets were transformed with `.transform()`

The number of features were reduced from 30 to 10 by selecting the 10 features that had the highest absolute correlation with Class on the training set only. This reduction is important for eLCS because the rule population must search a match set covering all the chosen features. Including useless features adds unnecessary search space and decreases rule specificity.

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

```
all_feature_cols = ['Time_scaled'] + [f'V{i}' for i in range(1, 29)] + ['Amount_transformed']

train_with_target = X_train_raw[all_feature_cols].copy()
train_with_target['Class'] = y_train.values

corr = train_with_target.corr()['Class'].drop('Class').abs().sort_values(ascending=False)
top_features = corr.head(10).index.tolist()

print("Selected features (top 10 by |correlation| with Class, computed on the training set only):")
print(corr.head(10))
```

Python

```
Selected features (top 10 by |correlation| with Class, computed on the training set only):
V17    0.335744
V14    0.274755
V12    0.245138
V3     0.216917
V10    0.215567
V7     0.200548
V16    0.194406
V11    0.143383
V1     0.121631
V5     0.121011
Name: Class, dtype: float64
```

The 10 chosen features were transformed in five quantile-based bins using KBinsDiscretizer fit on the training data only. This choice is supported by the fact that eLCS rules can be better combined with discrete, categorical-like conditions than arbitrary continuous sub-ranges. SMOTE was then applied only to the training set, synthesizing more fraud examples. This will result in significantly more examples of fraud for the rules discovered

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

Task 3 Results: Data Preprocessing and Feature Engineering

The raw dataset was deduplicated, leaving 283,726 rows, and then sub-sampled to 40,000 rows before splitting it into a train and test set, using the exact same per-class-count method as in Task 2. All preprocessing steps-Yeo-Johnson transformation on the Amount variable, Robust Scaler on the Time variable, correlation-based feature selection, quantile discretization, and SMOTE-were fitted only on the training set and then applied to the test set to ensure that no information from the test set could influence the decisions. The skewness of the Amount variable was reduced from 19.248 to 0.020 after the transformation.

The top ten features, ranked by absolute correlation with the Class variable, were selected as V17, V14, V12, V3, V10, V7, V16, V11, V1, and V5, and these were each converted into five quantiles.

SMOTE was applied only on the training set to over-sample the fraud class from 54 to 31,946 (out of a total of 63,892 rows). Finally, running the untouched eLCS model-using the same default parameters as in Task 2-on this pre-processed dataset indicates that pre-processing alone might be able to improve performance.

Preprocessed results:

Metric	Value
Accuracy	0.9782
Balanced Accuracy	0.9507
Precision	0.0649
Recall	0.9231
F1-score	0.1212

Preprocessing has clearly led to considerable performance gains over the Task 2 baseline. Balanced accuracy jumped from 0.5000 to 0.9507, and recall jumped from 0.0000 to 0.9231, implying that the model is now likely to correctly detect most fraud in the test set. Precision fell to 0.0649, a classic drawback of full SMOTE balancing, where the model will be over-eager to mark transactions as fraudulent given a baseline fraud prevalence of only 0.17%.

Running the original eLCS after preprocessing, feature selection and discretization, but with the 1:1 SMOTE balancing applied, no hyperparameter changes reveal just preprocessing has some positive impact on the results (Recall jumps from 0.0000 to 0.9231). But this increase in Recall happens at the expense of precision (0.0649). This, in turn, is likely due to the massive over-representation of the positive class, creating the rule population under the impression that fraudulent transactions are 50% of the population, far more than 0.17%.

Task 4: Improved LCS-Based System

Yaacoub:

```
from sklearn.utils import resample

#Subsample from the SMOTE-balanced training set for computational feasibility,
#keeping the 50/50 balance intact (Different from Task 2's subsample which kept the raw ~0.17% imbalance)
X_train_bal_sub, y_train_bal_sub = resample(X_train_resampled, y_train_resampled, n_samples=15000, stratify=y_train_resampled, random_state=42)

print(f"Balanced training subsample: {X_train_bal_sub.shape}, Fraud cases: {y_train_bal_sub.sum()}")
```

Balanced training subsample: (15000, 30), Fraud cases: 7500

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

```
from sklearn import elcs
import time
from sklearn.metrics import accuracy_score, balanced_accuracy_score, precision_score, recall_score, f1_score

#Improved system: same elcs algorithm, but trained on preprocessed + balanced data,
#with increased learning iterations (hyperparameter tuning)
improved_model = elcs(learning_iterations=20000, random_state=42)

start = time.time()
improved_model.fit(X_train_bal_sub.values, y_train_bal_sub.values)
elapsed = time.time() - start

#Evaluate on the UNTOUCHED, real-world-imbalanced test set
preds_improved = improved_model.predict(X_test_final.values)

print(f"Training time: {elapsed:.2f} seconds")
print(f"Accuracy: {accuracy_score(y_test, preds_improved):.4f}")
print(f"Balanced Accuracy: {balanced_accuracy_score(y_test, preds_improved):.4f}")
print(f"Precision: {precision_score(y_test, preds_improved, zero_division=0):.4f}")
print(f"Recall: {recall_score(y_test, preds_improved, zero_division=0):.4f}")
print(f"F1 Score: {f1_score(y_test, preds_improved, zero_division=0):.4f}")
```

Training time: 76.97 seconds
Accuracy: 0.9837
Balanced Accuracy: 0.9183
Precision: 0.0817
Recall: 0.8526
F1 Score: 0.1490

```
top_features = ['V17', 'V14', 'V12', 'V10', 'V11', 'V4', 'Time_scaled', 'Amount_scaled']

#Use the SAME subsampled rows as the full-feature model, just the reduced columns
#This keeps the comparison fair. (Identical training examples, only feature count differs)
X_train_bal_sub_selected = X_train_bal_sub[top_features]
X_test_selected_final = X_test_final[top_features]

print(f"Reduced-feature training subsample: {X_train_bal_sub_selected.shape}")
print(f"Reduced-feature test set: {X_test_selected_final.shape}")
```

Reduced-feature training subsample: (15000, 8)
Reduced-feature test set: (56746, 8)

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

```
improved_model_selected = eLCS(learning_iterations=20000, random_state=42)

start = time.time()
improved_model_selected.fit(X_train_bal_sub_selected.values, y_train_bal_sub.values)
elapsed = time.time() - start

preds_selected = improved_model_selected.predict(X_test_selected_final.values)

print(f"Training time: {elapsed:.2f} seconds")
print(f"Accuracy: {accuracy_score(y_test, preds_selected):.4f}")
print(f"Balanced Accuracy: {balanced_accuracy_score(y_test, preds_selected):.4f}")
print(f"Precision: {precision_score(y_test, preds_selected, zero_division=0):.4f}")
print(f"Recall: {recall_score(y_test, preds_selected, zero_division=0):.4f}")
print(f"F1 Score: {f1_score(y_test, preds_selected, zero_division=0):.4f}")
```

```
Training time: 42.27 seconds
Accuracy: 0.9879
Balanced Accuracy: 0.9099
Precision: 0.1053
Recall: 0.8316
F1 Score: 0.1870
```

The improved system combines three changes relative to the original baseline (Task 2): class-imbalance handling via SMOTE (applied only to the training set), preprocessing designed for LCS (scaling Time and Amount to match the standardised PCA components), and increased hyperparameter tuning (learning_iterations raised from 10,000 to 20,000). A second variant was also tested, additionally applying feature selection to reduce the feature set from 30 to 8 (the six PCA components most correlated with Class from Phase 1 plus Time_scaled and Amount_scaled).

The baseline's complete failure to detect fraud (recall = 0) was tied to eLCS's rule-discovery process having too few minority-class examples to evolve effective rules. SMOTE addresses this by balancing the training data. Scaling Time and Amount ensures these features are not overshadowed by the standardised PCA components during rule matching. Increased learning iterations allow the rule population more chances to converge given the larger, balanced training set. Feature selection, in the second variant, reduces the search space, possibly allowing rules to specialise more efficiently around the features already known to carry the strongest signal.

Both improved variants were evaluated on the same untouched, real-world-imbalanced test set used in Task 2, ensuring a fair comparison.

Metric	Baseline	Improved(30 features)	Improved(8 features)
Training Time	39.41s	76.97s	42.27s
Accuracy	0.9983	0.9837	0.9879
Balanced Accuracy	0.5000	0.9183	0.9099
Precision	0.0000	0.0817	0.1053
Recall	0.0000	0.8526	0.8316

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

F1-score	0.0000	0.1490	0.1870
----------	--------	--------	--------

Both improved variants display an improvement over the baseline: balanced accuracy rose from chance level (0.50) to over 0.90, and recall rose from 0.00 to over 0.83, meaning the improved system now properly identifies the majority of fraudulent transactions in the test set, addressing the false-negative cost concern raised in Phase 1's motivation. This comes at the cost of reduced precision, a known and expected trade-off of SMOTE-based rebalancing, since the model becomes more willing to flag transactions as fraudulent.

Matthew:

Improvement 1 – Partial SMOTE ratio: a `sampling_strategy=0.5` was used rather than the 1.0 SMOTE complete balancing, such that the minority class would be oversampled until it reached 50% of the number of samples in the majority class. With balanced classes the rule population is biased towards fraud, and legitimate cases are both equally probable. Not reflecting the real situation and leading it to over-guess the fraud. The partial ratio however would still be give the LCS quite a lot more fraud cases to learn from than the 54 frauds from before.

```
smote_partial = SMOTE(sampling_strategy=0.5, random_state=42)
X_train_bal, y_train_bal = smote_partial.fit_resample(X_train_final, y_train)

print(f"Before SMOTE: {X_train_final.shape[0]} rows, {y_train.sum()} fraud")
print(f"After partial SMOTE (0.5): {X_train_bal.shape[0]} rows, {y_train_bal.sum()} fraud")
```

Before SMOTE: 32000 rows, 54 fraud
After partial SMOTE (0.5): 47919 rows, 15973 fraud

Improvement 2 – Hyperparameter tuning and Subsumption: The value of the hyperparameter `nu`, an exponent in the mapping function that converts a rule accuracy into fitness, was raised from 5 to 10 which implies that the accuracy will be mapped much stronger and hence the highly accurate rule will have higher fitness. Further, configuration flag `do_correct_set_subsumption` was enabled (if a rule newly inserted into the set can be covered/subsumed by an existing rule, it is not introduced in the population, and the set's covering rule receives an increment to its numerosity). Neither of these modifications require changing the LCS loop beyond simply changing

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

configuration parameters from the previous tasks.

```
model_improved = eLCS(  
    learning_iterations=10000,  
    nu=10,  
    do_correct_set_subsumption=True,  
    random_state=42  
)  
  
start = time.time()  
model_improved.fit(X_train_bal.values, y_train_bal.values)  
elapsed_improved = time.time() - start  
  
preds_improved = model_improved.predict(X_test_final.values)  
  
acc_improved = accuracy_score(y_test, preds_improved)  
bal_acc_improved = balanced_accuracy_score(y_test, preds_improved)  
prec_improved = precision_score(y_test, preds_improved)  
rec_improved = recall_score(y_test, preds_improved)  
f1_improved = f1_score(y_test, preds_improved)  
  
print(f"Training time: {elapsed_improved:.2f}s")  
print(f"Accuracy: {acc_improved:.4f}")  
print(f"Balanced Accuracy: {bal_acc_improved:.4f}")  
print(f"Precision: {prec_improved:.4f}")  
print(f"Recall: {rec_improved:.4f}")  
print(f"F1-score: {f1_improved:.4f}")
```

Python

```
Training time: 11.80s  
Accuracy: 0.9945  
Balanced Accuracy: 0.9204  
Precision: 0.2075  
Recall: 0.8462  
F1-score: 0.3333
```

Metric	Task 2 (raw)	Task 3 (preprocessing only)	Task 4 (improved)
Accuracy	0.9982	0.9782	0.9945
Balanced Accuracy	0.5000	0.9507	0.9204
Precision	0.0000	0.0649	0.2075
Recall	0.0000	0.9231	0.8462
F1-score	0.0000	0.1212	0.3333
Training time	36.33s	15.42s	11.80s

The application of a partial SMOTE ratio combined with an increased nu and correct-set subsumption more than tripled precision to over 0.20 compared to Task 3 (0.0649 vs 0.2075) with a modest reduction in recall (0.9231 to 0.8462) and F1-score increased almost threefold (0.1212 to 0.3333). This pipeline iteration seemed to successfully balance fraud detection against excessive classification of legitimate transactions as fraudulent.

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

Task 5: Experiments

Yaacoub

```
#Regenerate baseline predictions on the SAME test set used for the improved model,  
# using the raw (unscaled) columns in their original order to match the baseline model's training format  
raw_cols_order = ['Time'] + [f'V{i}' for i in range(1, 29)] + ['Amount']  
X_test_raw_for_baseline = X_test_raw[raw_cols_order]  
  
preds_baseline_full = model.predict(X_test_raw_for_baseline.values)  
baseline_probs = model.predict_proba(X_test_raw_for_baseline.values)[: , 1]  
  
print(f"Baseline predictions generated on full test set: {len(preds_baseline_full)}")
```

Baseline predictions generated on full test set: 56746

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

```
from sklearn.metrics import confusion_matrix, roc_auc_score, average_precision_score

#Baseline_probs already generated above
improved_probs = improved_model.predict_proba(X_test_final.values)[: , 1]

print("Baseline Confusion Matrix:")
print(confusion_matrix(y_test, preds_baseline_full))
print(f"Baseline ROC AUC: {roc_auc_score(y_test, baseline_probs):.4f}")
print(f"Baseline PR-AUC: {average_precision_score(y_test, baseline_probs):.4f}")

print("\nImproved (30-feature) Confusion Matrix:")
print(confusion_matrix(y_test, preds_improved))
print(f"Improved ROC AUC: {roc_auc_score(y_test, improved_probs):.4f}")
print(f"Improved PR-AUC: {average_precision_score(y_test, improved_probs):.4f}")
```

Baseline Confusion Matrix:

```
[[56648   3]
 [   95    0]]
```

Baseline ROC AUC: 0.6578

Baseline PR-AUC: 0.2223

Improved (30-feature) Confusion Matrix:

```
[[55740   911]
 [   14    81]]
```

Improved ROC AUC: 0.9656

Improved PR-AUC: 0.5834

```
from statsmodels.stats.contingency_tables import mcnemar
import numpy as np

#Build the 2x2 contingency table of correct/incorrect predictions between the two models
baseline_correct = (preds_baseline_full == y_test)
improved_correct = (preds_improved == y_test)

both_correct = np.sum(baseline_correct & improved_correct)
baseline_only = np.sum(baseline_correct & ~improved_correct)
improved_only = np.sum(~baseline_correct & improved_correct)
both_wrong = np.sum(~baseline_correct & ~improved_correct)

contingency_table = np.array([[both_correct, baseline_only], [improved_only, both_wrong]])

print("Contingency Table:")
print(contingency_table)

result = mcnemar(contingency_table, exact=True)
print(f"\nMcNemar's test statistic: {result.statistic}")
print(f"p-value: {result.pvalue}")
```

Contingency Table:

```
[[55740   908]
 [   81    17]]
```

McNemar's test statistic: 81.0

p-value: 1.017443492443091e-177

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

Validation Strategy: An 80/20 stratified train-test split was used, rather than k-fold cross-validation mainly due to the computational cost of LCS training and prediction (single training runs took 40-75 seconds even on subsampled data (Tasks 2 and 4)) and generating predictions for the full 56,746-row test set took approximately 25 minutes per model. K-fold cross-validation would multiply this cost by the number of folds, making it impractical within the project's time constraints. Stratification confirmed the severe class imbalance (0.167% fraud) was preserved proportionally in both the training and test sets.

Evaluation Metrics: Given the class imbalance seen in Phase 1, accuracy alone was deemed unsuitable, as it can appear high even when a model fails to detect any fraud (seen by the baseline's 99.83% accuracy despite zero recall). Balanced accuracy, precision, recall, and F1-score were prioritised, alongside ROC-AUC and PR-AUC, with emphasis on PR-AUC as the more informative metric on imbalanced data. Confusion matrices were also looked at, since they reveal the practical cost of each error type (missed fraud vs. false alarms) discussed as a key trade-off in Phase 1's motivation.

Metric	Baseline	Improved (30 features)
Confusion Matrix	[[56648, 3], [95, 0]]	[[55740, 911], [14, 81]]
ROC-AUC	0.6578	0.9656
PR-AUC	0.2223	0.5834

The baseline model's confusion matrix confirms it failed to detect any of the 95 fraudulent transactions in the test set (0 true positives). The improved model correctly identified 81 of these 95 cases, at the cost of 911 false positives among legitimate transactions. ROC-AUC improved from 0.658 (barely above the 0.5 chance level) to 0.966, and PR-AUC, the more appropriate metric given the class imbalance, improved from 0.222 to 0.583.

Statistical Test: McNemar's test was used to compare the paired predictions of the baseline and improved models on the identical test set, since it is suited to comparing two classifiers correct/incorrect outcomes on matched samples. The test produced a statistic of 81.0 and a p-value of 1.02×10^{-177} , far below the $\alpha = 0.05$ significance threshold. This provides strong statistical evidence that the improved system's performance differs majorly from the baseline, confirming that the observed gains are not attributable to chance.

Matthew:

```
print(f"Train shape: {X_train_raw.shape}, Fraud cases: {y_train.sum()} ({y_train.mean():.4%})")
print(f"Test shape: {X_test_raw.shape}, Fraud cases: {y_test.sum()} ({y_test.mean():.4%})")
```

Train shape: (32000, 32), Fraud cases: 54 (0.1688%)
Test shape: (8000, 32), Fraud cases: 13 (0.1625%)

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

Validation strategy: For this investigation, a simple 80/20 stratified train-test split was chosen rather than k-fold cross-validation. Two factors motivate this decision. First, and more practically, fitting an eLCS instance took up to around 36 seconds depending on parameter configuration. A k-fold cross-validation would increase that time k-fold, which, given the 6 model comparisons performed in Task 6, would likely be prohibitively slow.

The second, and more important, factor is the class imbalance. Even using only the 40,000-row subsample, there are only roughly 54-55 cases of fraud in the training set. Subdividing this already limited fraud class into k folds creates a high likelihood of certain folds containing essentially zero fraud for eLCS to train on, even with carefully chosen hyperparameters. A stratified split ensures all available fraud cases are effectively concentrated in one training set and one test set, and a fixed `random_state=42` was used throughout for reproducibility.

```
print(f"Train shape: {X_train_raw.shape}, Fraud cases: {y_train.sum()} ({y_train.mean():.4%})")
print(f"Test shape: {X_test_raw.shape}, Fraud cases: {y_test.sum()} ({y_test.mean():.4%})")
```

Train shape: (32000, 32), Fraud cases: 54 (0.1688%)
Test shape: (8000, 32), Fraud cases: 13 (0.1625%)

Evaluation metrics: Previously, accuracy, balanced accuracy, precision, recall, and F1-score were reported for all of Tasks 2-4. Here, additional metrics are reported: ROC-AUC, PR-AUC, and the confusion matrix. ROC-AUC can be used to summarize the quality of the ranking over the full range of decision thresholds and might be perceived as too optimistic if class imbalance is high, since the false positive rate is estimated over a large set of legitimate transactions. PR-AUC, on the other hand, can be seen to be more sensitive to the performance on the minority class (fraud) and is thus considered a more informative ranking score here due to the actual fraud rate of 0.17%.

Metric	Task 3 (preprocessing only)	Task 4 (improved)
Confusion matrix	[[7814, 173], [1, 12]]	[[7945, 42], [2, 11]]
ROC-AUC	0.9259	0.9626
PR-AUC	0.8771	0.8474

Task 4's confusion matrix shows far fewer false positives than Task 3 (42 vs 173), at the cost of one additional false negative (2 vs 1), consistent with the precision-for-a-little-recall trade described in Task 4. Task 4 also has a higher ROC-AUC (0.9626 vs 0.9259), though Task 3 has a marginally higher PR-AUC (0.8771 vs 0.8474), reflecting that Task 3's ranking captures slightly more of the precision-recall trade-off in raw area terms even though its actual 0.5-threshold predictions are much noisier.

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

```
from sklearn.metrics import roc_auc_score, average_precision_score, confusion_matrix
```

```
proba_task3 = model.predict_proba(X_test_final.values)[: , 1]  
proba_task4 = model_improved.predict_proba(X_test_final.values)[: , 1]
```

```
roc_auc_task3 = roc_auc_score(y_test, proba_task3)  
roc_auc_task4 = roc_auc_score(y_test, proba_task4)  
pr_auc_task3 = average_precision_score(y_test, proba_task3)  
pr_auc_task4 = average_precision_score(y_test, proba_task4)
```

```
cm_task3 = confusion_matrix(y_test, preds)  
cm_task4 = confusion_matrix(y_test, preds_improved)
```

```
print("Task 3 (preprocessing only):")  
print(f" ROC-AUC: {roc_auc_task3:.4f}")  
print(f" PR-AUC: {pr_auc_task3:.4f}")  
print(f" Confusion matrix (rows=actual, cols=predicted):\n{cm_task3}")
```

```
print("\nTask 4 (improved):")  
print(f" ROC-AUC: {roc_auc_task4:.4f}")  
print(f" PR-AUC: {pr_auc_task4:.4f}")  
print(f" Confusion matrix (rows=actual, cols=predicted):\n{cm_task4}")
```

Python

```
Task 3 (preprocessing only):  
ROC-AUC: 0.9259  
PR-AUC: 0.8771  
Confusion matrix (rows=actual, cols=predicted):  
[[7814 173]  
 [ 1 12]]
```

```
Task 4 (improved):  
ROC-AUC: 0.9626  
PR-AUC: 0.8474  
Confusion matrix (rows=actual, cols=predicted):  
[[7945 42]  
 [ 2 11]]
```

Statistical test: McNemar's test: McNemar's test was used to check whether Task 4's improvement over Task 3 is statistically significant rather than noise from one particular test-set split. It is the appropriate choice here rather than a t-test, because both models were evaluated on the exact same test instances; it works directly on the disagreements between the two classifiers, where Task 3 was correct and Task 4 was wrong, and vice versa. The null hypothesis (H_0) is that Task 3 and Task 4 have the same error rate on this test set; the alternative (H_1) is that their error rates differ, tested at $\alpha = 0.05$.

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

```
from statsmodels.stats.contingency_tables import mcnemar
import numpy as np

#Build the 2x2 contingency table of correct/incorrect predictions between the two models
task3_correct = (preds == y_test.values)
improved_correct = (preds_improved == y_test.values)

both_correct = np.sum(task3_correct & improved_correct)
task3_only = np.sum(task3_correct & ~improved_correct)
improved_only = np.sum(~task3_correct & improved_correct)
both_wrong = np.sum(~task3_correct & ~improved_correct)

contingency_table = np.array([[both_correct, task3_only],[improved_only, both_wrong]])

print("Contingency Table:")
print(contingency_table)

result = mcnemar(contingency_table, exact=True)
print(f"\nMcNemar's test statistic: {result.statistic}")
print(f"p-value: {result.pvalue}")
```

Python

Contingency Table:

```
[[7824    2]
 [ 132   42]]
```

McNemar's test statistic: 2.0

p-value: 8.307438982451885e-37

Of the 8,000 test cases, Task 4 corrected 132 errors found by Task 3 (these are almost all false positives), whereas Task 3 was correct in only 2 cases where Task 4 was wrong, producing a very lopsided pattern of disagreement. The p-value from McNemar's exact test is $8.31\text{e-}37$, very much less than $\alpha = 0.05$, so the null hypothesis is rejected. This indicates that the difference between Task 4 and Task 3 is statistically significant on this test set and is unlikely to be explained by chance.

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

Task 6: Model Comparison

Yaacoub:

```
import time

print(f"Starting fit at {time.strftime('%H:%M:%S')}", flush=True)
model_preprocessed = eLCS(learning_iterations=10000, random_state=42)

start = time.time()
model_preprocessed.fit(X_train_bal_sub.values, y_train_bal_sub.values)
elapsed = time.time() - start
print(f"Fit finished at {time.strftime('%H:%M:%S')} (took {elapsed:.2f}s)", flush=True)

print(f"Starting predict at {time.strftime('%H:%M:%S')}", flush=True)
predict_start = time.time()
preds_preprocessed = model_preprocessed.predict(X_test_final.values)
predict_elapsed = time.time() - predict_start
print(f"predict finished at {time.strftime('%H:%M:%S')} (took {predict_elapsed:.2f}s)", flush=True)

print(f"Starting predict_proba at {time.strftime('%H:%M:%S')}", flush=True)
proba_start = time.time()
probs_preprocessed = model_preprocessed.predict_proba(X_test_final.values)[: , 1]
proba_elapsed = time.time() - proba_start
print(f"predict_proba finished at {time.strftime('%H:%M:%S')} (took {proba_elapsed:.2f}s)", flush=True)

print(f"\nTraining time: {elapsed:.2f} seconds")
print(f"Accuracy: {accuracy_score(y_test, preds_preprocessed):.4f}")
print(f"Balanced Accuracy: {balanced_accuracy_score(y_test, preds_preprocessed):.4f}")
print(f"Precision: {precision_score(y_test, preds_preprocessed, zero_division=0):.4f}")
print(f"Recall: {recall_score(y_test, preds_preprocessed, zero_division=0):.4f}")
print(f"F1-score: {f1_score(y_test, preds_preprocessed, zero_division=0):.4f}")
```

```
Starting fit at 18:48:54
Fit finished at 18:49:29 (took 35.05s)
Starting predict at 18:49:29
predict finished at 18:54:35 (took 306.72s)
Starting predict_proba at 18:54:35
predict_proba finished at 18:59:43 (took 307.27s)
```

```
Training time: 35.05 seconds
Accuracy: 0.9884
Balanced Accuracy: 0.9154
Precision: 0.1110
Recall: 0.8421
F1-score: 0.1961
```

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier

conventional_models = {
    'Random Forest': RandomForestClassifier(random_state=42, n_estimators=100),
    'Logistic Regression': LogisticRegression(random_state=42, max_iter=1000),
    'Decision Tree': DecisionTreeClassifier(random_state=42)
}

conventional_results = {}

for name, clf in conventional_models.items():
    start = time.time()
    clf.fit(X_train_resampled, y_train_resampled) # full SMOTE-balanced training set (not subsampled(these models scale fine))
    elapsed = time.time() - start

    preds = clf.predict(X_test_final)
    probs = clf.predict_proba(X_test_final)[:, 1]

    conventional_results[name] = {
        'time': elapsed,
        'accuracy': accuracy_score(y_test, preds),
        'balanced_accuracy': balanced_accuracy_score(y_test, preds),
        'precision': precision_score(y_test, preds, zero_division=0),
        'recall': recall_score(y_test, preds, zero_division=0),
        'f1': f1_score(y_test, preds, zero_division=0),
        'roc_auc': roc_auc_score(y_test, probs),
        'pr_auc': average_precision_score(y_test, probs)
    }

print(f"{name}: {conventional_results[name]}")

Python
Random Forest: {'time': 409.0227663993835, 'accuracy': 0.9994713283755683, 'balanced_accuracy': 0.8736312547091193, 'precision': 0.922077922077922, 'recall': 0.7473684210526316, 'f1': 0.8255813953488372, 'roc_auc': 0.969088760452967}
Logistic Regression: {'time': 1.119502067505918, 'accuracy': 0.9735487963909351, 'balanced_accuracy': 0.9237002366288884, 'precision': 0.052798902188295165, 'recall': 0.8736842105263158, 'f1': 0.09958008308320337, 'roc_auc': 0.959159}
Decision Tree: {'time': 49.620131909451904, 'accuracy': 0.9975681105270143, 'balanced_accuracy': 0.8464003903735615, 'precision': 0.37714285714285717, 'recall': 0.6947368421052632, 'f1': 0.4888888888888889, 'roc_auc': 0.8464003903735}

print(f"Original LCS (preprocessed) ROC-AUC: {roc_auc_score(y_test, probs_preprocessed):.4f}")
print(f"Original LCS (preprocessed) PR-AUC: {average_precision_score(y_test, probs_preprocessed):.4f}")

Original LCS (preprocessed) ROC-AUC: 0.9668
Original LCS (preprocessed) PR-AUC: 0.5506
```

The proposed improved LCS-based system was compared against the original LCS on both raw and preprocessed data, and against three conventional non-deep-learning models (Random Forest, Logistic Regression, and Decision Tree), all trained on the same SMOTE-balanced training data and evaluated on the untouched, real-world-imbalanced test set.

Model	Training Time	Accuracy	Balanced Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Original LCS (raw)	39.41s	0.9983	0.5000	0.0000	0.0000	0.0000	0.6578	0.2223
Original LCS (pre-processed)	35.05s	0.9884	0.9154	0.1110	0.8421	0.1961	0.9668	0.5506
Improved LCS (30 features)	76.97s	0.9837	0.9183	0.0817	0.8526	0.1490	0.9656	0.5834
Random Forest	409.02s	0.9995	0.8736	0.9221	0.7474	0.8256	0.9691	0.8122
Logistic Regression	1.12s	0.9735	0.9237	0.0528	0.8737	0.0996	0.9592	0.6829

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

Decision Tree	49.62s	0.9976	0.8464	0.3771	0.6947	0.4889	0.8464	0.2625
---------------	--------	--------	--------	--------	--------	--------	--------	--------

The original LCS on raw data (Task 2) failed completely, with balanced accuracy at chance level (0.500) and zero recall. Applying preprocessing alone (SMOTE, scaling) to the original, unmodified LCS system produced an improvement, with balanced accuracy rising to 0.9154 and recall to 0.8421 which is nearly matching the fully improved system's performance (balanced accuracy 0.9183, recall 0.8526). This indicates that preprocessing, rather than the hyperparameter tuning applied in Task 4, was the primary force of performance improvement; the gain from tuning alone was small by comparison.

Among the conventional models, Random Forest was the standout performer, achieving the highest accuracy (0.9995), by far the best precision (0.9221), and the best ROC-AUC (0.9691) and PR-AUC (0.8122) of all six models tested, though at a substantially higher training cost (409.02 seconds) than the other conventional models. Logistic Regression achieved the highest recall of any model tested (0.8737) but the lowest precision (0.0528), reflecting its simple linear decision boundary combined with SMOTE-balanced training data, which biases it toward predicting fraud. Decision Tree performed moderately across all metrics, without excelling on any single one.

Overall, the LCS-based models (preprocessed and improved) achieved a similar profile to Logistic Regression — high recall, low precision — while Random Forest achieved a better balance between the two, and the best overall discriminative ability (PR-AUC). This suggests that, for this dataset, conventional methods such as Random Forest may offer superior raw predictive performance compared to LCS-based approaches, though LCS retains an advantage in interpretability through its evolved rule sets.

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

Matthew:

```
n_total = 40000
fraud_rate = df_raw_lcs['Class'].mean()
n_fraud = round(n_total * fraud_rate)
n_legit = n_total - n_fraud

fraud_sample_raw = df_raw_lcs[df_raw_lcs['Class'] == 1].sample(n=n_fraud, random_state=42)
legit_sample_raw = df_raw_lcs[df_raw_lcs['Class'] == 0].sample(n=n_legit, random_state=42)
df_sample_raw = pd.concat([fraud_sample_raw, legit_sample_raw]).sample(frac=1, random_state=42).reset_index(drop=True)

X_raw6 = df_sample_raw.drop(columns=['Class']).values
y_raw6 = df_sample_raw['Class'].values

X_train_raw6, X_test_raw6, y_train_raw6, y_test_raw6 = train_test_split(
    X_raw6, y_raw6, test_size=0.2, stratify=y_raw6, random_state=42
)

model_raw_baseline = eLCS(learning_iterations=10000, random_state=42)
start = time.time()
model_raw_baseline.fit(X_train_raw6, y_train_raw6)
time_raw_baseline = time.time() - start

preds_raw_baseline = model_raw_baseline.predict(X_test_raw6)
proba_raw_baseline = model_raw_baseline.predict_proba(X_test_raw6)[:, 1]

print(f"Training time: {time_raw_baseline:.2f}s")
print(f"Accuracy: {accuracy_score(y_test_raw6, preds_raw_baseline):.4f}")
print(f"BALANCED Accuracy: {balanced_accuracy_score(y_test_raw6, preds_raw_baseline):.4f}")
print(f"Precision: {precision_score(y_test_raw6, preds_raw_baseline, zero_division=0):.4f}")
print(f"Recall: {recall_score(y_test_raw6, preds_raw_baseline, zero_division=0):.4f}")
print(f"F1-score: {f1_score(y_test_raw6, preds_raw_baseline, zero_division=0):.4f}")

from sklearn.metrics import roc_auc_score, average_precision_score
roc_auc_raw_baseline = roc_auc_score(y_test_raw6, proba_raw_baseline)
pr_auc_raw_baseline = average_precision_score(y_test_raw6, proba_raw_baseline)
print(f"ROC-AUC: {roc_auc_raw_baseline:.4f}")
print(f"PR-AUC: {pr_auc_raw_baseline:.4f}")
```

Python

```
Training time: 35.64s
Accuracy: 0.9982
Balanced Accuracy: 0.5000
Precision: 0.0000
Recall: 0.0000
F1-score: 0.0000
ROC-AUC: 0.8084
PR-AUC: 0.2435
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB

conventional_models = {
    'Random Forest': RandomForestClassifier(random_state=42, n_estimators=100),
    'k-Nearest Neighbours': KNeighborsClassifier(n_neighbors=5),
    'Gaussian Naive Bayes': GaussianNB()
}

conventional_results = {}

for name, clf in conventional_models.items():
    start = time.time()
    clf.fit(X_train_cont_bal, y_train_cont_bal)
    elapsed = time.time() - start

    preds_clf = clf.predict(X_test_cont)
    proba_clf = clf.predict_proba(X_test_cont)[:, 1]

    conventional_results[name] = {
        'time': elapsed,
        'accuracy': accuracy_score(y_test, preds_clf),
        'balanced_accuracy': balanced_accuracy_score(y_test, preds_clf),
        'precision': precision_score(y_test, proba_clf, zero_division=0),
        'recall': recall_score(y_test, proba_clf, zero_division=0),
        'f1': f1_score(y_test, proba_clf, zero_division=0),
        'roc_auc': roc_auc_score(y_test, proba_clf),
        'pr_auc': average_precision_score(y_test, proba_clf)
    }

    print(f"({name}): {conventional_results[name]}")

Random Forest: {'time': 14.4126191392212, 'accuracy': 0.999125, 'balanced_accuracy': 0.9227639144378866, 'precision': 0.6079, 'recall': 0.8461538461538461, 'f1': 0.7586286806053724, 'roc_auc': 0.9572574664599285, 'pr_auc': 0.8418886953617183}
k-Nearest Neighbours: {'time': 0.254775802287596, 'accuracy': 0.9985, 'balanced_accuracy': 0.924492781539232, 'precision': 0.2972972972972973, 'recall': 0.8461538461538461, 'f1': 0.44, 'roc_auc': 0.9086379597815357, 'pr_auc': 0.530516363822942}
Gaussian Naive Bayes: {'time': 0.40241312534807613, 'accuracy': 0.981, 'balanced_accuracy': 0.913866619807699, 'precision': 0.86832298136461963, 'recall': 0.8461538461538461, 'f1': 0.1243678104919541, 'roc_auc': 0.990157713387136, 'pr_auc': 0.1678271314933216}
```

Python

Three conventional models were chosen to span different modelling assumptions rather than three variants of the same idea: Random Forest (an ensemble of decision trees, a strong standard tabular-fraud-detection baseline capturing non-linear feature interactions without much tuning), k-Nearest Neighbours (instance-based and non-parametric, an interesting contrast to LCS since both are, in different ways, similarity/matching-based approaches), and Gaussian Naive Bayes (fast and probabilistic, a simple low-cost baseline revealing whether the dataset needs anything more sophisticated than treating features independently). All three were trained on the same 10 correlation-selected features used by the LCS models, but in continuous,

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

scaled form (prior to discretisation) with SMOTE reapplied, keeping the feature set identical across all six systems so differences reflect the modelling approach rather than a different data representation.

Model	Train Time	Acc.	Bal. Acc.	Prec.	Recall	F1	ROC-AUC	PR-AUC
Original LCS (raw)	36.33s	0.9982	0.5000	0.0000	0.0000	0.0000	0.8084	0.2435
Original LCS (preprocessed)	15.42s	0.9782	0.9507	0.0649	0.9231	0.1212	0.9259	0.8771
Improved LCS	11.80s	0.9945	0.9204	0.2075	0.8462	0.3333	0.9626	0.8474
Random Forest	14.41s	0.9991	0.9228	0.6875	0.8462	0.7586	0.9573	0.8419
k-Nearest Neighbours	0.15s	0.9965	0.9214	0.2973	0.8462	0.4400	0.9606	0.5105
Gaussian Naive Bayes	0.02s	0.9810	0.9137	0.0683	0.8462	0.1264	0.9302	0.1678

The original LCS on raw data failed almost completely at its default decision threshold, with balanced accuracy at chance level and zero recall, though its ROC-AUC (0.8084) shows some weak internal ranking signal that simply never crossed the 0.5 threshold given how few fraud examples it saw during training. Preprocessing alone fixed this dramatically, and the further tuning applied in Task 4 mainly traded some of that recall for a large gain in precision.

Among the conventional models, Random Forest was the clear standout, achieving the best precision (0.6875), the best F1-score (0.7586) of any model tested including all three LCS variants, and accuracy competitive with the improved LCS, all while matching the same recall (0.8462). k-Nearest Neighbours and Gaussian Naive Bayes both matched the 0.8462 recall of the improved LCS almost exactly (a product of the same SMOTE-balanced training data pushing every model toward similar sensitivity), but neither matched Random Forest's precision; Naive Bayes in particular performed worse than even the unmodified preprocessed LCS on precision and F1-score, consistent with its strong independence assumption being a poor fit for these PCA-derived, likely-correlated features.

Overall, Random Forest outperformed every LCS variant tested on precision and F1-score, while achieving comparable recall and balanced accuracy. Training time does not favour Random Forest as clearly: it trains faster than the raw and preprocessed LCS baselines, but not faster than the already-tuned improved LCS, so its advantage over

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

the strongest LCS variant is in predictive performance rather than speed. This suggests that for this specific dataset and feature set, a conventional ensemble method can outperform LCS-based approaches on raw predictive performance. LCS's main advantage that a black-box ensemble like Random Forest cannot offer is direct interpretability through its evolved rule set.

Task 7: Interpretation of Results

Yaacoub:

```
# Export the rules from the improved LCS model
improved_model.export_final_rule_population(X_test_final.columns, 'Class', 'improved_lcs_rules.csv', DCAL=True)

import pandas as pd
rules_df = pd.read_csv('improved_lcs_rules.csv')
print(f"Total rules in population: {len(rules_df)}")
rules_df.head(10)
```

Total rules in population: 923

	Specified Values	Specified Attribute Names	Class	Fitness	Accuracy	Numerosity	Avg Match Set Size	TimeStamp GA	Iteration Initialized	Specificity	Deletion Probability	Correct Count	Match Count
0	[7.557829845340796, 8.167993219290056], [-9.797...	V1, V2, V7, V13, V19, V20, V21, V28	1.0	0.100000	1.000000	1	13.747745	147	147	0.266667	0.000070	0	0
1	[-21.196268684838224, 74424659140688], [-13.04...	V2, V3, V6, V8, V12, V15, V20, V22, V23, V24...	0.0	0.481359	0.863960	1	149.874930	19989	310	0.400000	0.000788	4363	5050
2	[-34.18799585270649, -1.6906699560703657], [-31...	V1, V7, V8, V11, V13, V14, V16, V19, V24, V26	1.0	1.000000	1.000000	1	156.697110	19851	476	0.333333	0.000797	203	203
3	[-6.15816382983245, 5.824311687983606], [-3.958...	V4, V11, V12, V16, V21, V26	0.0	0.436105	0.847068	1	135.268016	19989	897	0.200000	0.000672	8912	10521
4	[-0.3843244770789568, 0.5979767836253294], [-34...	Time_scaled, V1, V5, V8, V10, V13, V17, V20, V...	1.0	1.000000	1.000000	1	74.996778	19851	1609	0.466667	0.000381	78	78
5	[-0.7904976999555993, 1.1013417651045074], [5.3...	Time_scaled, V2, V10, V12, V16, V17, V18, V19...	1.0	1.000000	1.000000	2	63.491154	19708	1629	0.366667	0.000645	79	79
6	[6.031157727835486, 17.522129518465704], [-31.4...	V4, V5, V8, V11, V13, V18, V19, V20, V21, V24...	1.0	1.000000	1.000000	1	64.723681	19708	1629	0.433333	0.000329	58	58
7	[-0.7904976999555993, 1.1013417651045074], [5.3...	Time_scaled, V2, V10, V12, V13, V16, V17, V18...	1.0	1.000000	1.000000	2	56.183049	19708	2548	0.333333	0.000571	63	63
8	[-12.79089014289112, 11.885684028320451], [-9.0...	V1, V9, V10, V11, V17, V18, V19, V23	1.0	1.000000	1.000000	1	205.961240	19677	3286	0.266667	0.001047	412	412
9	[-34.77189395818248, -21.477533364104065], [1.3...	V1, V4, V10, V11, V12, V19, V21, V23, V27	1.0	0.977274	0.995413	1	146.293982	19812	4555	0.300000	0.000744	217	218

```
# Filter for fraud-predicting rules with genuine evidence of usefulness (Correct Count > 0)
fraud_rules = rules_df[(rules_df['Class'] == 1) & (rules_df['Correct Count'] > 0)]
fraud_rules_sorted = fraud_rules.sort_values(by=['Correct Count', 'Fitness'], ascending=False)
print(f"Fraud-predicting rules with match evidence: {len(fraud_rules_sorted)}")
fraud_rules_sorted.head(5)[['Specified Attribute Names', 'Class', 'Fitness', 'Accuracy', 'Correct Count', 'Match Count', 'Specificity']]
```

Fraud-predicting rules with match evidence: 475

	Specified Attribute Names	Class	Fitness	Accuracy	Correct Count	Match Count	Specificity
16	V2, V6, V14, V18, V19, V23, V26, V27	1.0	0.993266	0.998650	4437	4443	0.266667
43	V12, V16, V19, V21, V23, V24, V28	1.0	0.667089	0.922225	4162	4513	0.233333
78	V2, V13, V14, V20, V25, V26	1.0	0.965945	0.993094	3020	3041	0.200000
38	V4, V5, V12, V16, V19, V21	1.0	0.959156	0.991694	2985	3010	0.200000
57	V3, V6, V8, V16, V24, V27, V28, Amount_scaled	1.0	0.487527	0.866163	2867	3310	0.266667

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

```
# Pull the full detail (including specified value ranges) for the top rules
top_rules_detail = rules_df.loc[[16, 43, 78, 38, 57]]
[['Specified Values', 'Specified Attribute Names', 'Class', 'Fitness', 'Accuracy', 'Correct Count', 'Match Count']]

for idx, row in top_rules_detail.iterrows():
    print(f"\n--- Rule {idx} ---")
    print(f"Attributes: {row['Specified Attribute Names']}")
    print(f"Values: {row['Specified Values']}")
    print(f"Class: {row['Class']}, Fitness: {row['Fitness']:.4f}, Accuracy: {row['Accuracy']:.4f}")
    print(f"Correct: {row['Correct Count']}/{row['Match Count']}")

--- Rule 16 ---
Attributes: V2, V6, V14, V18, V19, V23, V26, V27
Values: [-10.720110871879386, 20.604508519568896], [-14.777901339700112, 5.203740211152267], [-19.579160194071836, -2.783661063584087], [-9.666175782977124, 0.2762117723919816], [-2.0658576920881493, 5.333443661878903], [-10.1774120864272]
Class: 1.0, Fitness: 0.9933, Accuracy: 0.9986
Correct: 4437/4443

--- Rule 43 ---
Attributes: V12, V16, V19, V21, V23, V24, V28
Values: [-17.61892537540003, 1.4996840132899973], [-22.602616911891655, 3.5978002619297964], [-2.0658576920881493, 5.046382082872625], [-16.379121329407173, 17.171875008359546], [-10.515637058198214, 11.531254767937922], [-2.09618373517]
Class: 1.0, Fitness: 0.9671, Accuracy: 0.9222
Correct: 4162/4513

--- Rule 78 ---
Attributes: V2, V3, V14, V20, V25, V26
Values: [-24.067405425837215, 19.170726598864128], [-3.0621539078596504, 2.1539634804000017], [-28.17795353864001, -3.2439875480081586], [-6.005449439217071, 9.826993928514102], [-0.9924183345184782, 1.9175871974523289], [-1.41669667838087]
Class: 1.0, Fitness: 0.9659, Accuracy: 0.9931
Correct: 3828/3841

--- Rule 38 ---
Attributes: V4, V5, V12, V16, V19, V21
Values: [2.967496597135459, 9.836354589894051], [-28.927974674822686, 17.59784034079316], [-22.117218326734925, -1.4996840132899973], [-22.602616911891655, 3.5978002619297964], [-2.364680808476414, 5.046382082872625], [-16.37912132940717]
Class: 1.0, Fitness: 0.9592, Accuracy: 0.9917
Correct: 2985/3030

---
Attributes: V3, V6, V8, V16, V24, V27, V28, Amount_scaled
Values: [-25.652049707221884, -1.6601456288640204], [-13.485042447322416, 8.13444831478816], [-10.636909050087079, 9.989434846788855], [-20.81657392506366, 4.375224327380164], [-0.7574967597708981, 2.4617479396131987], [-3.450452583943802]
Class: 1.0, Fitness: 0.4875, Accuracy: 0.8662
Correct: 2807/3310
```

Interpreting the Model Comparison: Task 6 showed that preprocessing was the primary force of improvement for LCS-based models, with the original LCS on preprocessed data almost matching the fully improved system's balanced accuracy and recall. Compared with conventional models, Random Forest achieved the best overall precision-recall balance and discriminative ability (PR-AUC = 0.8122), whilst LCS-based models shared a similar profile to Logistic Regression: high recall, low precision. This insinuates Random Forest may offer better raw predictive performance on this dataset, but LCS offers predictions that can be traced back to a specific, human-readable rule.

Example LCS Rules: Three rules were taken from the improved LCS system's final rule population (923 total rules), selected for high match counts and high accuracy:

Rule 1 (matched 4,443 times, 99.86% accuracy): predicts fraud when V2 is between -10.72 and 20.60, V6 between -14.78 and 5.20, V14 between -19.58 and -2.78, V18 between -9.67 and 0.28, V19 between -2.07 and 5.33, V23 between -10.18 and 7.53, V26 between -1.01 and 0.85, and V27 between -1.17 and 5.32.

Rule 2 (matched 4,513 times, 92.22% accuracy): predicts fraud when V12 is between -17.62 and -1.50, V16 between -22.60 and 3.60, V19 between -2.07 and 5.05, V21 between -16.38 and 17.17, V23 between -10.52 and 11.53, V24 between -2.10 and 2.41, and V28 between -5.46 and 5.38.

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

Rule 3 (matched 3,041 times, 99.31% accuracy): predicts fraud when V2 is between -24.07 and 19.17, V13 between -3.06 and 2.15, V14 between -20.18 and -3.24, V20 between -6.01 and 9.83, V25 between -0.99 and 1.92, and V26 between -1.42 and 2.16.

Discussion of Decision-Making Process: Each rule represents a combination of conditions across several features. A transaction is classified as fraudulent only if all its feature values fall within every specified range. V14 appears as a narrow, negative range in both Rules 1 and 3 (approximately -20 to -3), consistent with Phase 1's correlation analysis, where V14 (-0.293) was among the strongest negative correlation of fraud. V12 appears as a negative range in Rule 2, matching its Phase 1 correlation (-0.251). V19 and V23 recur across multiple top rules with narrow ranges despite showing negligible linear correlation with Class in Phase 1 (0.034 and -0.006 respectively), meaning LCS may be capturing non-linear or interaction-based patterns that a simple correlation coefficient would not detect.

Supporting Interpretability: Unlike Random Forest or Logistic Regression, which make predictions through aggregated votes across many trees or one global weighted combination of all features respectively, each LCS rule is a readable "if-then" statement. A specific prediction can be explained by pointing to the exact rule (or rules) that matched the transaction and the exact feature ranges responsible which is a practical advantage in a fraud-detection context, where a flagged transaction may need to be explained to a customer, bank officer, or regulator.

Trustworthiness and Practical Meaningfulness: This advantage comes with an important issue. Because V1 through V28 are PCA-transformed components with no disclosed real-world meaning (Phase 1), a rule such as "V14 between -19.58 and -2.78" cannot be translated into a business-interpretable statement like "transactions from this merchant category" or "purchases above a certain amount at unusual hours." The rules are mathematically precise and traceable, but not semantically meaningful to a human stakeholder without access to the original, undisclosed features. This limits the value of LCS's interpretability advantage for this specific dataset — the explanation is sound, but the content of the explanation is not actionable in a real-world deployment sense.

Matthew:

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

```
rule_population_path = 'task7_rule_population.csv'
model_improved.export_final_rule_population(
    headerNames=np.array(top_features),
    className="Class",
    filename=rule_population_path,
    DCAL=True
)

rules_df = pd.read_csv(rule_population_path)
rules_df['Class'] = rules_df['Class'].astype(int)

print(f"Total macro-classifiers in the final population: {len(rules_df)}")
print(f"Class breakdown (all rules):\n{rules_df['Class'].value_counts()}")

# Keep only rules with enough training experience to be meaningful (seen as a correct
# match at least 10 times), rather than rare, barely-tested rules the GA has not yet pruned.
experienced_rules = rules_df[rules_df['Correct Count'] >= 10].copy()
print(f"\nRules with Correct Count >= 10: {len(experienced_rules)}")
print(f"Class breakdown (experienced rules):\n{experienced_rules['Class'].value_counts()}")

Total macro-classifiers in the final population: 673
Class breakdown (all rules):
Class
0    424
1    249
Name: count, dtype: int64

Rules with Correct Count >= 10: 557
Class breakdown (experienced rules):
Class
0    373
1    184
Name: count, dtype: int64

def describe_condition(rule):
    """Describe one rule's specified attributes and (domain-clipped) bin-range conditions as text."""
    attrs = [a.attrs for a in rule['Specified Attribute Names']].split(',')
    rule_values = rule['Specified Values']
    # Split on ":", while keeping the brackets, since each condition is "[low,high]"
    import re
    conditions = re.findall(r'\[([^\]]+)\]', rule_values)
    parts = []
    for attr, cond in zip(attrs, conditions):
        low, high = (float(a) for a in cond.split(','))
        low_c, high_c = max(low, 0.0), min(high, 4.0)
        parts.append(f'{attr} in bin-range [{low_c:.1f}, {high_c:.1f}]')
    return ' AND '.join(parts)

# A broad, single-attribute, perfectly accurate legitimate-transaction rule
rule_legit = experienced_rules[
    (experienced_rules['Class'] == 0) & (experienced_rules['Specificity'] == 0.1)
].sort_values('Match Count', ascending=False).iloc[0]

# A two-attribute fraud rule
rule_fraud_legit = experienced_rules[
    (experienced_rules['Class'] == 1) & (experienced_rules['Specificity'].round(2) == 0.2)
].sort_values('Match Count', ascending=False).iloc[0]

# A three-attribute fraud rule
rule_fraud_legit = experienced_rules[
    (experienced_rules['Class'] == 1) & (experienced_rules['Specificity'].round(2) == 0.3)
].sort_values('Match Count', ascending=False).iloc[0]

for label, rule in [
    ("Rule 1 (broad, legitimate-transaction)", rule_legit),
    ("Rule 2 (two-attribute, fraud)", rule_fraud_legit),
    ("Rule 3 (three-attribute, fraud)", rule_fraud_legit),
]:
    predicted = "fraud" if rule['Class'] == 1 else "legitimate"
    print(f"Label: {label}")
    print(f"IF {describe_condition(rule)}")
    print(f"then predict: {predicted}")
    print(f"Accuracy: {rule['Accuracy']:.4f} | Numerosity: {int(rule['Numerosity'])} | "
          f"Correct/Match count: {int(rule['Correct Count'])}/{int(rule['Match Count'])}")
    print()

Rule 1 (broad, legitimate-transaction)
IF x0 in bin-range [0.00, 4.00]
then predict: legitimate
Accuracy: 0.9998 | Numerosity: 1 | Correct/Match count: 5031/5040

Rule 2 (two-attribute, fraud)
IF x0 in bin-range [0.00, 0.70] AND x1 in bin-range [1.75, 4.00]
then predict: fraud
Accuracy: 0.7977 | Numerosity: 1 | Correct/Match count: 2828/3545

Rule 3 (three-attribute, fraud)
IF x0 in bin-range [0.00, 1.75] AND x1 in bin-range [0.00, 0.40] AND x2 in bin-range [1.00, 4.00]
```

Interpreting the main findings from the model comparison

Task 6 compared six systems on the same held-out test set (8,000 transactions, 13 fraud): the raw-data eLCS, the preprocessed eLCS, the improved eLCS, and three conventional models trained on the same 10 features. The raw-data eLCS baseline never predicted fraud at all, despite a respectable ROC-AUC of 0.8084 - the model can rank fraud cases somewhat above legitimate ones internally, but with only 54 fraud examples and no class balancing, it never had enough incentive during rule evolution to commit to predicting the fraud class. Preprocessing and balancing fixed recall almost completely (0.9231) at the cost of precision (0.0649), because full 1:1 SMOTE balancing made the rule population treat fraud as roughly as common as legitimate transactions. Task 4's partial SMOTE plus a sharper fitness exponent and correct-set subsumption shifted that trade-off back toward precision (0.2075, more than tripled) while giving up only a little recall (0.8462).

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

Against the conventional models, every one matches the improved eLCS on recall (0.8462), and Random Forest is well ahead on precision (0.6875 vs 0.2075) and F1 (0.7586 vs 0.3333). Training time does not follow the same pattern: Random Forest (14.41s) is faster than the raw and preprocessed LCS baselines (36.33s, 15.42s) but is not faster than the already-tuned improved LCS (11.80s), so its real advantage over the strongest LCS variant is predictive performance rather than speed. This is an expected outcome rather than a failure of the LCS approach specifically: ensemble tree methods are well suited to tabular, already-engineered features like PCA components, whereas eLCS's rule evolution must additionally discover useful attribute combinations from scratch through covering, mutation and crossover. The trade-off this project is interested in is predictive performance versus interpretability, which is why the improved eLCS model's rule population is examined directly below.

Extracting the rule population

skeLCS's eLCS exposes the trained rule population through `export_final_rule_population()`, which writes every macro-classifier to CSV with the real feature names substituted in. Each row records which attributes the rule tests, the condition on each specified attribute, the predicted class, and quality indicators: Accuracy, Numerosity, Correct Count, Match Count, and Specificity. The final population contained 673 macro-classifiers (424 predicting legitimate, 249 predicting fraud). Rules were filtered to those with Correct Count ≥ 10 , leaving 557 rules (373 legitimate, 184 fraud) with enough training evidence to be a genuine pattern rather than noise.

Three sample rules and how they drive predictions

The 10 features going into eLCS are quantile bin indices (0 = lowest 20% of training-set values, up to 4 = highest 20%) rather than raw PCA values. Because SMOTE interpolation after discretisation produces fractional bin values, eLCS ends up treating these attributes as continuous rather than discrete, representing each rule condition as a numeric range rather than an exact bin match; some raw bounds fall outside the true 0-4 domain due to eLCS's covering mechanism, so each bound is clipped to [0, 4] before being described below.

Rule 1 (broad, legitimate-transaction): IF V17 in bin-range [0.80, 4.00] THEN predict Legitimate. Accuracy 0.8930, correct on 5,031 of 5,634 matches. V17 was the single strongest fraud indicator among the 10 selected features by correlation (ranked 1st); fraudulent transactions in this dataset tend to take unusually low raw V17 values, which is exactly the bottom bin this rule excludes.

Rule 2 (two-attribute, fraud): IF V10 in bin-range [0.00, 0.78] AND V11 in bin-range [1.73, 4.00] THEN predict Fraud. Accuracy 0.7977, correct on 2,828 of 3,545 matches. V10 and V11 rank 5th and 8th by correlation among the 10 selected features - weaker individually than V17 or V14, so this rule relies on combining two moderately-informative signals rather than one dominant one, which shows up as a noticeably lower accuracy than Rule 1.

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

Rule 3 (three-attribute, fraud): IF V17 in bin-range [0.00, 1.12] AND V14 in bin-range [0.00, 0.66] AND V11 in bin-range [3.38, 4.00] THEN predict Fraud. Accuracy 0.9841, correct on 2,470 of 2,510 matches. This rule is both more specific (three attributes) and more accurate than Rules 1 and 2, reusing V17 and V11 alongside V14 (ranked 2nd by correlation) - all three independently known to carry fraud signal. This is the general pattern in the population: higher-specificity rules (3-4 attributes) reach higher accuracy than broader 1-2 attribute rules, at the cost of covering fewer training instances.

How Rule-base outputs support interpretability

Every eLCS prediction is supported by the rule(s) that went active for that one instance. Each rule has a short, human-readable conjunction of feature-range conditions, and its own track record, in terms of accuracy, numerosity, and match count. This structure is different from Random Forest, which generates a prediction by averaging hundreds of small paths through the forest of 100 trees. In Random Forest, there is no single, concise rule to explain why a fraud prediction was made for that transaction without a post-hoc explainability tool like SHAP. With eLCS, the explanation comes built in with the prediction, not with an added explainability step. Also, the rules are rated based on their inherent reliability statistics as opposed to being taken as equally trustworthy. For example, Rule 1 has a Correct Count in the thousands and high accuracy (0.89), so can reasonably be considered a real pattern. However, a rule seen only once or twice should not be treated as a reliable pattern, and the model distinguishes between them using these fields without needing to run an explainability test.

Are these explanations trustworthy and practically meaningful?

Yes, to a point, and with caveats. The evidence for belief is twofold: first, each of the three rules holds against thousands of training matches, and second, the features mentioned in those rules (V17, V14, V10, V11) were independently deemed the most predictive in Task 3. As described there, V17 and V14 were the top-two strongest correlates of fraud among all candidate features, and V10 and V11 held 5th and 8th place, respectively.

That these rules lean on top-performing attributes for detecting fraud suggests the rule population is converging on real predictive patterns rather than spurious connections (though it's still worth noting that Rule 2, the two-attribute rule, incorporates a weaker pair of inputs than Rules 1 and 3).

All three are also directionally consistent in their fraud correlation (V17-low, V14-low, V10-low, V11-high).

The caveats are threefold. First, because the entire input pool for the fraud class is limited - only 54 instances at the subsample count where training occurs - the absolute match counts are amplified via SMOTE (as SMOTE synthesizes fraudulent instances through neighbour interpolation). While this doesn't invalidate the pattern match, it means a count of, e.g., 5,000 matches relies on what those 54 original cases define, plus their SMOTE-neighbours, not an absolute diversity of 5,000 empirical ground-truth fraud cases.

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

Second, all features V1 through V28 are anonymised PCA components; thus, even if one is entirely convinced of the accuracy of Rule 1 (a low V17 being highly indicative of fraud), it is of little real-world actionability to a fraud analyst since the true semantic meaning of V17 remains undisclosed.

Third, and for reasons of both SMOTE's post-discretisation interpolation and eLCS's covering mechanism, one should read these rules as identifying bins rather than establishing an unbreakable physical barrier on the data range.

Summing these up, the rule population can reasonably be treated as a high-level descriptor of fraud behaviour in the PCA space, and is arguably more human-interpretable than the mechanisms of the Random Forest model, even if Random Forest is the objectively stronger prediction tool as demonstrated in Task 6. However, to transform these into hard, actionable operational thresholds for a fraud analyst, one would ideally need both the original, non-anonymised attribute names and a larger set of non-synthetic ground-truth fraud samples.

Task 8: Discussion, Limitations, and Future Work

Reasons for Performance Improvement: The improvement from the original LCS baseline (recall = 0.00) to the preprocessed and improved variants (recall > 0.83) was driven mainly by addressing the class imbalance using SMOTE, rather than by the hyperparameter tuning applied in the improved system. The original LCS on preprocessed data achieved balanced accuracy and recall almost identical to the fully improved system, meaning that eLCS's rule-discovery process was the main bottleneck: with only 378 fraud examples in the raw training set, the algorithm had a small amount of minority-class evidence to evolve rules covering fraud, regardless of how many learning iterations were allowed. This tells us that for genetic/evolutionary rule-based systems like LCS, data-level interventions (balancing class representation) may matter more than algorithm-level tuning when the imbalance is severe. This phenomenon was experienced similarly between the two pipelines independently: for instance, for the second pipeline (40,000-row subset), the application of SMOTE (without a subsequent hyperparameter change) pushed the recall from 0.0000 to 0.9231 directly.

Practical Meaning of the Results: Beyond the metrics, the improved system's practical value lies in the trade-off it represents, that being catching roughly 85% of fraudulent transactions at the cost of a substantially higher false-positive rate (911 false alarms among legitimate transactions) than Random Forest's higher-precision approach. In a real deployment, this trade-off would need to be weighed against the business cost of each error type. A missed fraud case may cost more in direct financial loss than a false alarm cost in customer inconvenience, but too high a false-positive rate risks eroding customer trust and needing more manual review teams. The LCS-based system's interpretability, shown through its evolved rules in Task 7, offers an advantage for justifying individual decisions, even though its predictive performance did not beat Random Forest's. Two different points on the precision-recall trade-off spectrum were reached by the two pipelines examined here. The 40,000-row pipeline

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

correctly captured about 85% of fraud (11 cases of fraud out of 13) while producing 42 false positives from the 7,987 legitimate transactions, a considerably tighter trade-off compared to the 911 false positives that the 15,000-row pipeline achieved (this comparison makes the sensitivity of this trade-off quite obvious, as it varies significantly with both the chosen SMOTE ratio and the chosen subset of training data).

Limitations: Many limitations affect the reliability and generalisability of these findings. Firstly, due to the high computational cost of LCS training and prediction, models were trained and evaluated on subsamples (15,000-40,000 rows, depending on the pipeline, for training) rather than the full dataset, which may not properly encapsulate the diversity of fraud patterns present in the complete 284,807-row dataset. Secondly, the severe class imbalance, while addressed through SMOTE, means synthetic fraud examples were generated through interpolation rather than representing new real-world fraud cases, which will not fully reflect the diversity of fraud tactics. Thirdly, the dataset's PCA-anonymisation, while needed for privacy, limits the practical interpretability of LCS's rule-based explanations, since feature conditions can't be mapped to real-world transactions. Finally, the dataset covers only a 48-hour window from September 2013, raising questions about whether patterns learned remain relevant to current fraud tactics, which always evolve.

Future Work: Many ideas could better this work. Testing alternative LCS variants (e.g., XCS, which uses accuracy-based fitness rather than eLCS's strength-based approach) could show whether rule-discovery improvements generalise beyond eLCS specifically. Alternative class-imbalance techniques, such as cost-sensitive learning or ensemble-based resampling methods (e.g., SMOTE-ENN), could be compared against the SMOTE approach used here to decide whether further gains in precision are achievable without sacrificing recall. External validation on a separate, more recent fraud dataset would help assess whether the evolved rules generalise beyond this specific dataset and timeframe. Finally, domain expert review. Essentially involving access to the original, non-anonymised transaction features which would be needed to properly decide whether LCS's rule-based explanations are actionable for fraud analysts in practice, addressing the interpretability limitation seen in Task 7.

Mohammed Yaacoub Abou Chlih 21138726

Matty Luriz 23206450

GitHub link: <https://github.com/ExplosiveSalad/Credit-Card-Skimmers.git>

References

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357. <https://doi.org/10.1613/jair.953>

Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research*, 18(1), 559–563. <https://dl.acm.org/doi/10.5555/3122009.3122026>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. <https://dl.acm.org/doi/10.5555/1953048.2078195>

UrbsLab/scikit-eLCS (Version 1.2.4) [Computer software]. (2022, May 12). GitHub. <https://github.com/UrbsLab/scikit-eLCS>

Urbanowicz, R. J., & Moore, J. H. (2015). ExSTraCS 2.0: Description and evaluation of a scalable learning classifier system. *Evolutionary Intelligence*, 8, 89–116. <https://doi.org/10.1007/s12065-015-0128-8>

Yeo, I.-K., & Johnson, R. A. (2000). A new family of power transformations to improve normality or symmetry. *Biometrika*, 87(4), 954–959. <https://doi.org/10.1093/biomet/87.4.954>

Machine Learning Group – ULB. (2017). *Credit Card Fraud Detection (Version 3) [Data set]*. Kaggle. <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

Deliverables

Each group must submit:

- Final report: 5,000--6,500 words, excluding references and appendices. **(Added and checked)**
Source code or Jupyter Notebook.
GitHub repository link and commit-history evidence. **(Added and checked)**
Contribution statement, including a brief individual reflection from each group member.
Cleaned and feature-engineered dataset, or clear dataset reproduction instructions.
Turnitin similarity report with a similarity score not exceeding 20%